Palestine Innovation Competition — 2025

*Problem Domain: Smart Cities & Community Infrastructure*

# MASARI

مساري

*Palestine's Comprehensive Mobility & Logistics Platform*

**PROJECT PROPOSAL**

**Team: Masari**

Zainab Atwa · Ghaidaa Masalmeh · Bashar Tamizeh

Kawther Atwa · Hiba Hawamdah · Sarah Abu Mandil

# 1 Project Overview

| | |
|---|---|
| **Project Title** | Masari (مساري) — Palestine's Comprehensive Mobility & Logistics Platform |
| **Team Name** | Team Masari |
| **Domain** | Smart Cities · Community Infrastructure · Transportation |
| **Team Members** | Zainab Atwa, Kawther Atwa, Sarah Abu Mandil, Ghaydaa Masalmeh, Heba Hawamda, Bashar Tamizeh |

## Team Roles & Background

| Name | Role | Background |
|---|---|---|
| **Zainab Atwa** | Team Lead & UI/UX | Software Engineering graduate; mobile dev & UI/UX experience |
| **Kawther Atwa** | Mobile Developer | CS student; specialist in mobile development (Flutter/Dart) |
| **Sarah Abu Mandil** | AI Engineer | AI engineering; RAG systems; Arabic NLP; ML integration |
| **Ghaydaa Masalmeh** | Mobile Developer | CES student; Flutter/Dart, UI/UX |
| **Hiba Hawamdah** | Backend & Integrations | CS student; systems integration; server-side logic |
| **Bashar Tamizeh** | Full-Stack Developer | Full-stack dev; frontend components; QA |

## Short Description

Masari is a mobile-first mobility and logistics platform that connects passengers, drivers, and merchants through a verified driver network, a live interactive map with real-time location sharing, and a flexible delivery system for both people and parcels.

The platform focuses on solving the missing layer in the Palestinian market: executing transportation and delivery in a structured, reliable, and real-time way.

Passengers can book licensed public drivers who are officially authorised and committed to fixed routes, fair pricing, and request reliability.

Once a ride is confirmed, both driver and passenger share live location on the map, eliminating confusion in pickup points and reducing delays.

Merchants and individuals can send parcels through direct driver coordination or batch delivery, allowing multiple shipments to be delivered in one trip at a significantly lower cost.

Masari integrates existing map and road-status APIs instead of rebuilding them, and focuses on what current tools do not provide: a trusted, operational system for moving people and goods in real time.

An AI agent is included as a supporting feature to assist with booking and matching, but the core value of the platform lies in its verified transport network and execution layer.

# 2 Problem Statement

## The Real-World Problem

The main issue is not transportation difficulty itself, nor the lack of maps or road information.

The real gap is the absence of a real-time execution system for transportation and delivery that connects users directly with verified drivers in a coordinated, trackable, and cost-efficient way.

While existing tools (such as mapping applications or communication apps) can show locations or provide general information, they do not support the actual execution of a trip or delivery in a structured system.

There is currently no unified platform that enables:

- Live matching between users and available drivers based on actual routes and timing
- Real-time coordination using a shared live map (Live Map feature) instead of manual location descriptions
- Verified driver-based booking with clear rules and accountability
- Structured pricing based on routes rather than informal negotiation
- End-to-end tracking of rides and deliveries from request to completion

## Who Is Affected & Why It Matters

- Merchants and small businesses that ship goods daily and suffer from high delivery costs due to per-parcel pricing, with no option for batch delivery, no flexible coordination with drivers, and limited shipment tracking or confirmation.
- Individual users who need occasional delivery services but have no direct way to request a driver to pick up and deliver items on demand at a reasonable cost.

- Drivers who lack a structured platform to offer their services, resulting in under-utilised capacity, especially for those who could participate in delivery but currently have no organised system to do so.
- Daily commuters who struggle to find reliable and committed drivers face frequent delays, unclear pickup points, and no real-time coordination with drivers.

### Supporting Evidence & Observations

| Problem Area | Observed Reality |
|---|---|
| Pickup & Location Confusion | Users must explain their location manually; drivers struggle to find them, leading to delays, missed pickups, and cancellations |
| No Real-Time Coordination | No shared live location between driver and user during booking, resulting in inefficient communication and poor user experience |
| High Delivery Costs | Delivery services charge per parcel, even when multiple items go to the same destination, increasing costs for merchants and individuals |
| Lack of Batch Delivery | No system allows grouping multiple parcels into one delivery, forcing users to pay separately for each shipment |
| No Direct Delivery Access | Individuals cannot easily request a driver to pick up and deliver items on demand |
| Driver Underutilisation | Many drivers travel with unused capacity and have no structured platform to offer delivery services |
| Transport Fragmentation | Booking rides or deliveries depends on personal networks (calls, WhatsApp), with no accountability or tracking |
| Lack of Tracking & Confirmation | No clear delivery status system (e.g. picked up, in transit, delivered), causing uncertainty for both sender and receiver |
| Unverified Passenger Transport | Passengers rely on informal and unverified drivers, with no guarantee of licensing, pricing consistency, or commitment to the trip |

## 3 Proposed Solution

### How Masari Solves the Problem

Masari is a single mobile application (iOS and Android) that unifies five pillars:

| Pillar | Feature | Problem It Solves |
|---|---|---|
| **Live Map & Coordination** | Real-time driver tracking + live location sharing | Eliminates pickup confusion and manual coordination |
| **Verified Driver System** | Licensed drivers + contract enforcement | Ensures trust, legal compliance, and pricing consistency |
| **Smart Delivery System** | Batch + direct delivery | Reduces delivery cost and improves efficiency |
| **AI Agent (Support)** | Assists booking & matching | Simplifies user interaction |
| **External Data Integration** | Road & checkpoint APIs | Provides awareness without rebuilding existing systems |

### Key Features

- Live Interactive Map with real-time location sharing between driver and user
- Verified Driver System (passenger + delivery roles)
- Route-based pricing enforcement for public drivers
- Batch parcel delivery for merchants (cost optimisation)
- On-demand delivery for individuals
- End-to-End Order Tracking:

  Accepted → Picked Up → In Transit → Delivered
- Delivery confirmation from both sender and receiver
- Geofencing notifications (driver approaching pickup point)
- Trust Score system (based on trips, punctuality, cancellations)
- Merchant dashboard for managing and tracking shipments

## 4 System Architecture & Technical Approach

### High-Level System Design

Masari follows a modular technical architecture designed to support a focused MVP around two core use cases:

1. Route-based passenger pickup
2. Merchant delivery batching

The system is built as a mobile-first platform supported by backend services, real-time communication, AI-based request understanding, route matching, delivery batching, and practical route-based optimization.

Masari does not attempt to rebuild existing map systems or replace the full transportation infrastructure. Instead, it focuses on the missing operational layer in the Palestinian market: connecting passengers, merchants, and verified drivers through a structured, trackable, and route-aware execution system.

The architecture separates the user interface, backend logic, AI service, matching engine, delivery batching logic, and real-time updates. This makes the platform easier to develop, test, improve, and scale gradually after the MVP is validated.

| Layer | Technology | Responsibility |
|---|---|---|
| **Mobile Client** | Flutter | Passenger, merchant, and driver interfaces; request creation; live status tracking; pickup and delivery screens |
| **Backend API** | Node.js + Express | Authentication, user management, driver profiles, request handling, order management, and business logic |
| **Database** | PostgreSQL | Users, drivers, passenger requests, merchant orders, parcels, routes, trust scores, and request history |
| **Real-Time Layer** | Firebase Realtime Database / Socket.io | Live request updates, driver availability, location sharing, delivery status updates, and pickup coordination |
| **AI Agent Service** | Python + FastAPI | Understanding informal user requests and extracting request type, pickup point, destination, time, parcel count, and route direction |
| **Route Matching Service** | Python or Backend Service | Matching passengers with verified drivers already moving in the same direction |
| **Delivery Batching Service** | Python Optimization Module | Grouping merchant parcels based on route direction, destination proximity, capacity, and delivery time |
| **Optimization Layer** | Classical Scoring + Heuristic Optimization | Comparing manual coordination, nearest-driver assignment, and Masari route-based matching |
| **Maps Layer** | OpenStreetMap or Existing Map APIs | Pickup points, destinations, driver routes, route visualization, and location display |
| **Merchant Dashboard** | React or Web View | Managing merchant orders, tracking delivery statuses, and viewing batch delivery results |

| | | |
|---|---|---|
| **Admin Panel** | Web Dashboard | Driver verification, unclear request review, pilot monitoring, safety reports, and KPI tracking |

## Architecture LayersData Flow

1. The passenger, merchant, or driver opens the Masari mobile application.
2. A passenger submits a pickup request by entering the current location, destination, and preferred time.
3. A merchant submits multiple delivery orders by entering pickup location, customer destinations, parcel size, delivery priority, and preferred time.
4. A driver activates a route by entering current route, direction, available seats, parcel capacity, and expected movement time.
5. The AI Agent receives the user input and converts it into structured operational data.
6. The Route Matching Service checks available drivers according to route direction, pickup distance, timing, available capacity, trust score, and route deviation.
7. The Delivery Batching Service groups compatible parcels according to destination proximity, route overlap, parcel size, delivery time, and driver capacity.
8. The Optimization Layer ranks possible matches using a practical scoring model.
9. The driver receives only route-compatible passenger or parcel requests.
10. Once the driver accepts the request, the system updates the passenger or merchant in real time.
11. The request status is tracked until completion.

## Core Technical Logic

Masari uses a route-based decision model rather than a simple nearest-driver model.

A normal system may choose the geographically closest driver. Masari chooses the most suitable driver based on:

Route direction
Pickup distance
Expected arrival time
Available seats
Parcel capacity
Trust score
Delivery compatibility
Route deviation
Passenger waiting time
Merchant delivery priority

This makes the system more suitable for the Palestinian context, where the closest driver is not always the best driver. A driver farther away may already be moving in the correct direction, may have available capacity, and may create less route disruption.

## AI Agent Role

The AI Agent is not used as a decorative chatbot. It supports the operational logic of the platform.

The AI Agent helps convert informal user input into structured data.

For example:

User input:

“بدي أركب من عند البوليتكنك على سيارة رايحة بيت لحم الساعة 3”

AI output:

Request type: Passenger pickup
Pickup point: Palestine Polytechnic University area
Destination: Bethlehem
Time: 3 PM
Route direction: Toward Bethlehem
Required action: Find verified drivers passing near the pickup point

Another example:

User input:

“عندي خمس طلبات رايحات على نفس المنطقة”

AI output:

Request type: Merchant delivery batching
Pickup point: Shop location
Number of parcels: Five
Destination pattern: Same or nearby area
Required action: Group compatible parcels and find a suitable driver

The AI Agent should also return a confidence score. If the confidence score is low, the system can ask the user to confirm the pickup point, destination, or request type before matching.

## Optimization Approach

Masari does not rely on experimental or unrealistic optimization technologies in the MVP.

The first version uses a practical classical scoring model and heuristic optimization approach. This is enough to prove the core value of the platform: selecting the most suitable driver based on route direction, timing, capacity, trust score, pickup distance, and delivery compatibility.

The system compares three assignment approaches:

| Method | Description |
|---|---|
| **Manual Coordination** | Similar to current WhatsApp or phone-based coordination |
| **Nearest Driver** | Chooses the closest driver geographically |
| **Masari Route-Based Optimizer** | Chooses the most suitable driver based on route compatibility, timing, capacity, trust score, and delivery logic |

This keeps the prototype realistic, technically credible, and achievable within the competition timeline.

### Why This Architecture Is Suitable

This architecture is suitable because Masari is not only a booking app. It is a route-based execution platform.

The mobile app handles user interaction.
The backend manages users, requests, drivers, and statuses.
The AI Agent understands messy real-life requests.
The matching service connects passengers with compatible drivers.
The batching service groups merchant parcels efficiently.
The optimization layer improves decision quality.
The real-time layer keeps users updated.
The admin panel adds control and safety during the pilot stage.

The modular structure also supports teamwork because each team member can own a clear technical component. This makes development more organized and makes the final demo stronger.

## 5 Implementation Plan

### Implementation Strategy

The implementation plan is built around a focused MVP. Masari will not attempt to build a full national mobility system in the first version. The goal is to prove that the platform can solve two practical Palestinian problems:

1. Students and commuters need route-based pickup without returning unnecessarily to central stations.
2. Merchants need to batch multiple deliveries through compatible driver routes instead of paying for separate deliveries.

The MVP will prove the core intelligence of Masari through AI request understanding, route-based matching, delivery batching, real-time tracking, verified drivers, and a comparison dashboard.

The project will be implemented in five phases over sixteen weeks.

## Phase 1: Foundation and Product Design

**Timeline** **Weeks 1–2**

### Deliverables

- Finalize the MVP scope.
- Confirm the two core use cases.
- Define passenger, merchant, and driver user flows.
- Design Figma screens for the mobile app.
- Define the database schema.
- Prepare sample route data.
- Prepare sample merchant delivery orders.
- Prepare Palestinian Arabic request examples.
- Set up GitHub repository.
- Set up backend and mobile project structure.
- Define API requirements.

### Milestone

Focused prototype design ready.

### Purpose

This phase prevents scope creep. It ensures that the team builds the core route-based pickup and delivery batching logic before adding secondary features.

---

## Phase 2: Core System Build

**Timeline** **Weeks 3–5**

### Deliverables

- Build Flutter mobile screens.
- Create passenger request flow.
- Create merchant delivery entry flow.
- Create driver route activation flow.
- Build backend APIs for users, drivers, requests, and orders.
- Set up a PostgreSQL database.
- Create a basic driver profile and verification structure.
- Implement basic request status updates.
- Connect the mobile app with backend APIs.
- Create initial admin monitoring structure.

### Milestone

Core request system ready.

### Purpose

This phase proves that passengers and merchants can submit requests, drivers can activate routes, and the system can store and update operational data.

## Phase 3: AI, Matching, and Delivery Batching

**Timeline Weeks 6–8**

### Deliverables

Build the AI Agent service using Python and FastAPI.
Configure the AI Agent using limited Palestinian Arabic examples.
Extract request type, pickup point, destination, time, passenger count, parcel count, and route direction.
Build Route Matching Service.
Build Delivery Batching Service.
Create basic driver ranking logic.
Create basic parcel grouping logic.
Add confidence score for unclear AI extraction.
Add human-in-the-loop review for uncertain cases.
Connect AI outputs with backend request flow.

### Milestone

Intelligent matching prototype ready.

### Purpose

This phase builds the intelligence core of Masari. The AI Agent converts informal requests into structured data, while the matching and batching services turn that data into operational decisions.

## Phase 4: Optimization and Comparison Dashboard

**Timeline Weeks 9–11**

### Deliverables

- Build manual coordination baseline.
- Build nearest-driver assignment baseline.
- Build Masari route-based optimizer.
- Refine the route-based scoring model.
- Create a comparison dashboard.
- Compare matching time, route deviation, waiting time, driver utilization, and delivery batching efficiency.
- Test passenger pickup scenario.
- Test merchant batch delivery scenario.
- Show match explanation for recommended drivers.

### Milestone

Measurable intelligent prototype ready.

### Purpose

This phase makes Masari competition-ready because it shows measurable improvement. Judges should see that Masari is not only a user interface. It is a better route-based decision engine.

## Phase 5: Testing and Demo Preparation

**Timeline Weeks 12–16**

### Deliverables

- Conduct controlled testing with students, merchants, and drivers.
- Test passenger pickup from a known route point.
- Test merchant delivery batching with multiple orders.
- Test live request tracking.
- Test AI Agent extraction accuracy.
- Test driver acceptance flow.
- Fix bugs.
- Improve UI based on feedback.
- Clean demo data.
- Prepare presentation slides.
- Prepare recorded or live demo.
- Prepare the final proposal.
- Prepare pilot feedback summary.

### Milestone

Competition-ready MVP.

### MVP Scope

The MVP will include:

Route-based passenger pickup.
Merchant delivery batching.
Driver route activation.
AI request understanding.
Route matching.
Delivery batching.
Live request tracking.
Trust-based driver profile.
Basic admin review.
Comparison dashboard.

The MVP will not include:

In-app payment.
QR payment.
Fuel station live availability.
Full emergency volunteer mode.
Public transport schedule integration.
Municipality dashboard.
Large-scale multi-city deployment.
Full e-commerce API integration.
Advanced monetization.

These features are deferred because the first version must prove the core matching and optimization logic before expanding.

# 6 Team Skills & Role Distribution

| Member | Role | Key Skills | Contributes To |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **Zainab Atwa** | Team Lead + UI/UX | Flutter, UI/UX Design, Product Management, Mobile Dev | App UI, overall architecture, sprint planning |
| **Kawther Atwa** | Mobile Developer | Flutter/Dart, state management, mobile app architecture | Full mobile client (iOS & Android) |
| **Sarah Abu Mandil** | AI Engineer | Python, RAG pipelines, Arabic NLP, LangChain, ML model tuning | AI chatbot, dispatch engine, pricing model |
| **Ghaydaa Masalmeh** | Backend Developer | Node.js, PostgreSQL, REST API design, database schema | Backend APIs, database, business logic |
| **Heba Hawamda** | Backend + Integrations | Firebase, Redis, real-time systems, server-side integration | Real-time engine, WebSockets, live map data |
| **Bashar Tamizeh** | Full-Stack + QA | React/Node, testing frameworks, code review, deployment | Merchant dashboard, QA, App Store deployment |

## Skill Gaps & Mitigation

| Skill Gap | Risk | Mitigation |
|---|---|---|
| **Advanced production-level AI** | AI may misunderstand informal Palestinian Arabic requests | Start with limited domain examples, rule-based fallback, and confidence scoring |
| **Real-time GPS reliability** | Location sharing may be unstable in weak-connectivity areas | Use lightweight updates, last-known location, and status-based fallback |
| **Large route dataset** | Full real route data may not be available early | Use realistic simulated data and manually collected pilot routes |
| **Optimization complexity** | Advanced optimization may be difficult in the MVP period | Start with weighted scoring and heuristic matching |
| **Driver adoption** | Drivers may hesitate to join early | Begin with a small verified pilot group and simple driver interface |

| | | |
|---|---|---|
| **QA and deployment** | Bugs may appear during final demo | Dedicate the final phase to testing, demo data cleanup, and controlled scenario testing |

# 7 Feasibility Analysis

## Why This Is Achievable in 4 Months

- Technology Maturity & API Integration: We are utilizing a production-proven stack (Flutter, Node.js, Firebase). By integrating existing local APIs (like Azmeh or Tareeqi) instead of building data layers from scratch, we significantly accelerate the development of our RAG-based AI Agent.
- Phased Delivery (MVP by Week 11): Our roadmap is clear. The first 6 weeks focus on the Core Logic and API connectivity, followed by the Agent's decision-making engine. This leaves a full month for rigorous testing of the "Live Map" and "Cargo Pooling" features.
- Focused & Realistic Scope: Masari is strategically limited to the West Bank and a specific dual-use case (Public Transport + Consolidated Cargo). This avoids "Scope Creep" and ensures the AI Agent is highly specialized in local logistics.
- Specialized Team Coverage: Each technical layer (Mobile, Backend, AI Orchestration, UI/UX) is handled by a dedicated member. As an AI Engineer, my focus is on the Agent’s orchestration and RAG retrieval, ensuring no single point of failure in the system's intelligence.

## Risks & Mitigation

| Risk | Mitigation Strategy |
|---|---|
| AI chatbot accuracy in dialect | Start with rule-based fallback; iteratively improve RAG accuracy using real user queries collected in pilot. |
| Real-time GPS reliability in WB | Offline map tile caching; graceful degradation to last-known position when connectivity is poor. |
| Driver onboarding / cold-start | Partner with 20–30 pilot drivers in one city (e.g., Ramallah) before full launch to seed the map with real data. |
| Scope creep | MVP explicitly excludes: payment gateway, QR delivery confirmation, multi-language support — deferred to post-competition. |

| | |
|---|---|
| Data privacy | No PII stored beyond what is needed; GDPR-aligned data practices; driver ID verified but not permanently stored in plain text. |

### What We Will NOT Build (Scope Limits)

to ensure the stability and reliability of Masari's core logistics engine within the 12 week timeframe , we have strategically deferred the following modules:

- In-app payment processing (deferred — using cash on delivery or external transfer for pilot).
- QR-code payment suspension system (designed, not implemented in MVP).
- Multi-city scaling infrastructure (pilot is single city; architecture supports expansion post-competition).
- Web application (mobile-only for MVP).

## 8 Innovation & Impact

### What Makes Masari Unique

| Innovation | Why No Competitor Does This |
|---|---|
| **Checkpoint-aware routing** | Google Maps and Waze have no real-time data on Palestinian checkpoint status. Masari's crowdsourced layer is community-verified and hyper-local. |
| **Palestinian dialect AI assistant** | No existing platform offers a natural-language Arabic booking interface tuned to Palestinian spoken dialect. |
| **Cooperative relay cargo** | No Palestinian logistics platform allows a driver passing a pickup point to relay cargo to the primary delivery driver — reducing idle capacity. |
| **Trust Score beyond stars** | Composite driver score (trips, punctuality, cancellations, ratings) provides far richer signal than a simple 1–5 star average. |
| **Emergency Mode (zero commission)** | A platform feature that activates during total closures to connect volunteers with those in need — no commercial platform offers this. |
| **Fuel station live availability** | Community or owner-confirmed fuel status on the map — critical during shortages that are common in the West Bank. |

## Expected Impact

- Commuters: reduced time wasted at unexpected checkpoints; instant access to verified, nearby drivers on their exact route.
- Merchants: lower delivery cost through batch pricing; full shipment visibility from pickup to delivery; ability to upload 50 orders at once.
- Drivers: increased income through better utilisation; cooperative relay reduces dead-leg trips.
- Community safety: community-powered alerts create a collective early-warning system that benefits every road user.
- Economic: estimated to reduce per-delivery cost by 20–40% for merchants using batch cargo features; drivers could increase utilisation by 30%+.

## Business Model & Revenue Strategy

Masari is designed as a sustainable platform business. The following revenue streams ensure the platform can operate and grow beyond the competition:

| Revenue Stream | Mechanism | Target Audience |
|---|---|---|
| **Route Advertising** | Sponsored placements for businesses (shops, restaurants, services) located on high-traffic routes — shown contextually on the live map. | Local businesses on key corridors |
| **Platform Commission** | A small percentage fee on each completed ride or delivery transaction, deducted transparently from the driver's payout. | All platform users |
| **Merchant Subscriptions** | Monthly / annual plans for high-volume merchants: priority dispatch, advanced analytics dashboard, bulk-order API access, dedicated support. | SMEs & daily shippers |
| **Driver Verification (one-off)** | A one-time document-processing fee paid by drivers at onboarding to cover ID and licence verification costs. | Drivers joining the platform / new drivers |

| | | |
|---|---|---|
| **Data Insights (future)** | Anonymised, aggregated movement and logistics data sold to urban planners, NGOs, and municipal authorities for infrastructure planning — no PII shared. | Institutional partners |

Emergency Mode and volunteer matching are always commission-free — this is a core product commitment, not dependent on revenue pressure.

# 9 Validation Plan

## How We Will Validate Masari

Masari will be validated by testing whether it actually improves the two main problems it was designed to solve:

1. Route-based passenger pickup
2. Merchant delivery batching

The validation will not depend only on general user satisfaction. It will compare Masari against current manual coordination and simple nearest-driver logic.

This is important because the value of Masari is not just that users can submit a request. The real value is that the system can make a better route-based decision.

## Validation Logic

Masari will compare three methods:

| Method | Description |
|---|---|
| **Manual Coordination** | Similar to current WhatsApp, phone calls, and personal contact coordination |
| **Nearest Driver** | Chooses the geographically closest driver, even if the route is not optimal |
| **Masari Optimizer** | Chooses based on route direction, timing, capacity, trust, route deviation, and compatibility |

This comparison shows why Masari is different from a normal booking app. The nearest driver is not always the best driver. Masari chooses the most suitable driver.

## 9.1 Passenger Pickup Validation

### Scenario

A student is near Palestine Polytechnic University and wants to go to Bethlehem around 3 PM.

### Current Method

The student may go back to the central station.
The student may ask in WhatsApp groups.
The student may call several drivers.
The student may waste time and money.
There is no clear tracking or pickup confirmation.

### Masari Method

The student opens Masari.
The student enters pickup point, destination, and preferred time.
The AI Agent understands the request.
The Route Matching Service finds drivers already moving toward Bethlehem.
The system ranks drivers based on route direction, pickup distance, timing, available seats, trust score, and route deviation.
The student selects a verified driver.
The driver receives the pickup point.
Both sides track the request status.

### Metrics

Time saved
Passenger waiting time
Extra cost avoided
Pickup clarity
Driver route compatibility
Route deviation
Trust score relevance
User satisfaction

## 9.2 Merchant Delivery Validation

### Scenario

A shop owner has five orders going to nearby destinations.

### Current Method

The shop owner calls several drivers.
Each order may be sent separately.
Delivery cost is high.
There is no batching logic.
Tracking is unclear.
The customer may not receive clear confirmation.

### Masari Method

The shop owner enters all orders into Masari.
The AI Agent structures the delivery data.
The Delivery Batching Service groups compatible parcels.
The optimization logic suggests the most suitable driver.
The driver picks up the batch.
The merchant tracks each parcel status.
The system confirms delivery.

### Metrics

Cost per parcel
Number of parcels served per route
Number of calls reduced
Delivery tracking completion
Driver capacity utilization
Delivery time
Merchant satisfaction

## 9.3 AI Agent Validation

The AI Agent will be tested using Palestinian Arabic and informal user requests.

### Example Test Inputs

بدي أركب من عند البوليتكنك على سيارة رايحة بيت لحم
بدي أبعت طلبين لنفس المنطقة
في سائق مارق من عند الحرس
عندي ثلاث طرود رايحات على باب الزاوية
بدي سيارة من يطا للخليل الساعة ثلاثة

### The AI Agent Must Correctly Identify

Request type
Passenger or delivery
Pickup point
Destination

Time
Passenger count
Parcel count
Route direction
Possible urgency

### AI Success Metric

The AI Agent should correctly understand at least 85% of common pilot examples.

If confidence is low, the system should ask the user to confirm the unclear detail before matching.

## 9.4 Optimization Validation

Masari will validate its optimization logic by comparing the result of three assignment methods.

| Method | Expected Weakness |
|---|---|
| Manual Coordination | Slow, unstructured, difficult to track |
| Nearest Driver | May ignore route direction, capacity, trust, or delivery compatibility |
| Masari Optimizer | Uses route direction, timing, capacity, trust, and route compatibility |

### Optimization Metrics

Matching time
Route deviation
Estimated cost
Passenger waiting time
Number of served requests
Driver capacity utilization
Parcel batching efficiency
Trust score impact

The goal is not to test advanced theoretical optimization. The goal is to prove that a structured route-based scoring model gives better decisions than manual coordination or distance-only matching.

## 9.5 Controlled Pilot Testing

After internal testing, Masari will be tested in a limited pilot area.

### Pilot Participants

20–30 verified drivers
50–100 passengers
5–10 local merchants
A limited number of repeated routes
A limited number of known pickup points

### Suggested Pilot Routes

Hebron to Bethlehem
Yatta to Hebron
Palestine Polytechnic University area to Bethlehem
Bab Al Zawiya to nearby areas
Central station routes

The pilot should remain limited so the team can measure performance accurately and avoid overextending the MVP.

## 9.6 Success Targets

| Metric | Target |
|---|---|
| **Passenger matching time** | Less than 2 minutes in demo scenario |
| **AI request understanding** | At least 85% on test examples |
| **Delivery tracking completion** | At least 95% status continuity |
| **Batch delivery efficiency** | Lower cost than individual delivery |
| **Driver utilization** | More requests served per route |
| **User satisfaction** | 4 out of 5 or higher |
| **Driver verification** | 100% of active pilot drivers verified |
| **Pickup clarity** | Clear reduction in pickup confusion |
| **Route deviation** | Kept within acceptable route limits |

### Validation Statement

Masari will be validated by proving that it saves time for passengers, reduces delivery cost for merchants, improves driver utilization, and provides better coordination than current WhatsApp and phone-based methods.

The validation dashboard should make the value visible by comparing manual coordination, nearest-driver logic, and Masari route-based optimization.

## 10 Future Work

### Post-Competition Roadmap

Future work will expand Masari only after validating the two core use cases:

1. Route-based passenger pickup
2. Merchant delivery batching

The first version must prove that passengers and parcels can be matched with existing driver routes in a structured, trusted, and trackable way.

Masari's future development should not be presented as a list of random features. It should be presented as a phased expansion from a focused MVP into a broader Palestinian smart mobility and logistics infrastructure layer.

### Short-Term Future Work

In the short term, Masari will improve the MVP based on pilot feedback.

### Priorities

Improve AI Agent understanding of Palestinian Arabic.
Improve pickup point recognition using local landmarks.
Improve route matching accuracy.
Improve delivery batching logic.
Add more repeated local routes.
Add admin review panel for uncertain matches and user reports.
Add offline queuing for weak internet areas.
Improve merchant dashboard.
Improve delivery confirmation.
Improve trust score calculation.
Support recurring rides for students and employees.
Improve driver onboarding and verification workflow.

### Purpose

The short-term goal is to make the MVP more reliable, not larger. These improvements directly support the two core use cases.

### Mid-Term Future Work

In the medium term, Masari can expand to additional Palestinian cities and route corridors after the first pilot is validated.

### Priorities

Expand to additional city routes.
Add more verified drivers.
Create partnerships with universities.
Create partnerships with local shops.
Create partnerships with driver groups and transport offices.
Add bulk order upload for merchants.
Add scheduled delivery.
Add route scheduling for repeated-route drivers.
Add QR delivery confirmation if operationally ready.
Add simple payment options after legal and financial review.
Improve optimization using historical data.
Develop stronger merchant analytics.

### Purpose

The medium-term goal is to increase adoption and operational value while keeping the platform focused on route-based mobility and delivery.

### Long-Term Future Work

In the long term, Masari can become a broader Palestinian smart mobility and logistics infrastructure platform.

### Priorities

Integrate shared taxi and public transport schedules.
Support intercity delivery coordination.
Build institutional dashboards for municipalities.
Provide anonymized aggregated mobility insights.
Add emergency mobility support during closures.
Add fuel station status if relevant and data is reliable.
Develop advanced optimization experiments.

Build a broader Palestinian smart mobility infrastructure layer.
Support planning insights for universities, municipalities, NGOs, and business associations.

## Purpose

The long-term goal is to move from a student prototype to a scalable Palestinian infrastructure layer. This must happen gradually and only after the matching and optimization engine is validated.

## Advanced Optimization Research

After validating the MVP with real users and real route data, Masari can explore more advanced optimization techniques.

This may include:

Advanced assignment algorithms
Predictive route demand modeling
Historical route pattern analysis
Multi-request optimization
Dynamic batching models
Quantum-inspired optimization experiments

Quantum-inspired optimization will not be part of the MVP. It will be treated only as a future research direction that may help improve large-scale passenger-driver-parcel assignment when the platform expands to more cities, more drivers, and more delivery requests.

This approach keeps the current prototype realistic while leaving space for advanced technical development in the future.

## Future Data and Ethics Direction

One important future direction is the use of anonymized and aggregated mobility data to support planning decisions.

Municipalities and institutions could use these insights to understand:

Route pressure
Delivery demand
Transportation gaps
Recurring pickup points
Repeated road disruption patterns
Areas with weak driver availability

However, Masari must not sell individual movement data. In the Palestinian context, location privacy is not only a technical issue. It is a personal safety requirement.

All future data insights must be anonymized, aggregated, and non-identifiable.

## Strategic Growth Direction

| Phase | Focus |
|---|---|
| **Phase 1** | Stabilize the MVP and validate one pilot area |
| **Phase 2** | Expand verified drivers and merchant users |
| **Phase 3** | Add scheduling, stronger batching, and analytics |
| **Phase 4** | Expand across Palestinian cities and corridors |
| **Phase 5** | Become a Palestinian smart mobility and logistics infrastructure layer |

## Future Work Statement

Masari's future development will move from a focused student prototype to a scalable Palestinian mobility infrastructure layer, but only after validating the core matching and optimization engine.

The future of Masari is not based on copying foreign transportation models. It is based on improving Palestinian daily mobility behavior through AI, trust, tracking, and route-based optimization.

**MASARI — Moving Palestine Forward**

*We are not adapting a foreign product — we are building Palestinian infrastructure, by Palestinians, for Palestinians.*

Zainab Atwa · Ghaydaa Masalmeh · Bashar Tamizeh · Kawther Atwa · Hiba Hawamdeh · Sarah Abu Mandil